

When Tabular Boosting Beats Deep Learning on Signal Features

A data-scaling study of GBM, TabPFN, and deep models

2i

2026-07-29

Abstract

There is a common assumption that deep learning eventually beats classical tabular models on tasks where features can be derived from physical principles, such as wireless signal classification. This paper turns that assumption into one narrow, testable question and measures it directly. On the 6dB subset of the public RadioML2016.10a dataset, an 11-class modulation classification task, we attach 28 hand-engineered cumulant and spectral features and compare gradient-boosting families (LightGBM, XGBoost, CatBoost, Random Forest) against modern tabular learners (TabPFN v2, RealMLP, TabM, AutoGluon) and a deep learning model that consumes raw IQ samples directly (a mixture-of-experts transformer reproduction), all on the same train/test split. In the small-data regime with only 539 training examples, LightGBM reached an accuracy of 0.855, ahead of every competitor, while the raw-IQ deep model reached only 0.681. Scaling the training set from 539 to 2,000 examples raised the deep model’s accuracy from 0.368 to 0.581, narrowing the gap to gradient boosting from 36.0 to 21.3 percentage points. That narrowing trend is real, but reruns of the deep model at 5,000 and 9,900 training examples did not complete, so whether a crossover point exists within this dataset’s range, and where, remains unverified. The honest conclusion cuts both ways: on small-to-medium physically-engineered feature sets, tabular boosting is the practical winner, and that advantage widens further once interpretability and training cost are taken into account. But nothing in this experiment supports the much stronger claim that this advantage holds indefinitely as data grows without bound.

Contents

1	Abstract	3
2	Why Tabular Boosting: Interpretability, Small Data, and the Edge	3
3	Method: Features and Model Families	3
4	Data and Experimental Setup	4
5	Results	5
5.1	Table 1: Accuracy by Model (539 training examples, on 28 physical features, 1,100 test examples)	5
5.2	Table 2: The GBM-Deep Learning Gap by Data Scale	5
6	Limitations: The Crossover Needs to Be Revisited at Larger Scale	6
7	Reproducibility	7
8	References	8

1 Abstract

There is a common assumption that deep learning eventually beats classical tabular models on tasks where features can be derived from physical principles, such as wireless signal classification. This paper turns that assumption into one narrow, testable question and measures it directly. On the 6dB subset of the public RadioML2016.10a dataset, an 11-class modulation classification task, we attach 28 hand-engineered cumulant and spectral features and compare gradient-boosting families (LightGBM, XGBoost, CatBoost, Random Forest) against modern tabular learners (TabPFN v2, RealMLP, TabM, AutoGluon) and a deep learning model that consumes raw IQ samples directly (a mixture-of-experts transformer reproduction), all on the same train/test split. In the small-data regime with only 539 training examples, LightGBM reached an accuracy of 0.855, ahead of every competitor, while the raw-IQ deep model reached only 0.681. Scaling the training set from 539 to 2,000 examples raised the deep model’s accuracy from 0.368 to 0.581, narrowing the gap to gradient boosting from 36.0 to 21.3 percentage points. That narrowing trend is real, but reruns of the deep model at 5,000 and 9,900 training examples did not complete, so whether a crossover point exists within this dataset’s range, and where, remains unverified. The honest conclusion cuts both ways: on small-to-medium physically-engineered feature sets, tabular boosting is the practical winner, and that advantage widens further once interpretability and training cost are taken into account. But nothing in this experiment supports the much stronger claim that this advantage holds indefinitely as data grows without bound.

2 Why Tabular Boosting: Interpretability, Small Data, and the Edge

Problems built on time series from physical sensors, wireless signal classification, industrial vibration diagnostics, medical biosignal discrimination, tend to share three constraints. First, labeled training data is genuinely scarce. Collecting tens of thousands of examples of a specific fault waveform or a specific modulation scheme is rare in the field; models usually have to be built from hundreds to a few thousand labeled examples. Second, decisions must be explainable. In domains where a wrong call is costly, telecom regulation, industrial safety, medical certification, an answer to “why was this signal classified as this modulation” has to be expressible as feature importance or a decision boundary before it clears an approval process. Third, inference frequently has to run at the edge, on small embedded devices or low-power gateways rather than a server, in real time.

These three constraints favor gradient-boosted (GBM) tabular models over deep learning. Features derived from physical principles, such as a signal’s higher-order cumulants or spectral moments, already compress much of the class-discriminative information by construction, so tree ensembles find stable decision boundaries from only a few hundred training samples. Deep learning that consumes raw signals, by contrast, must learn feature extraction itself from data, which requires far more samples to extract the same information. Feature importance from a tree ensemble is also more direct and stable than post-hoc explanation techniques for deep networks; an engineer can immediately verify which physical quantity actually contributed to a decision. And because inference in a boosted tree is nothing more than a chain of conditional branches, it finishes in milliseconds even on a low-power microcontroller with no GPU.

Rather than accept this assumption at face value, this paper measures three things on one concrete task. First, given the same physical features, where do GBM, modern tabular learners, and a raw-signal deep model actually rank in the small-data regime. Second, how quickly does that ranking change as training data grows. Third, given that observation, what can we confidently claim, and where should we say we still do not know. The third point matters most. Benchmark papers commonly stop at showing the point where their method wins. This paper reports the point where it wins, the point where it does not, and the point that remains unanswered, with equal weight.

3 Method: Features and Model Families

The task is to classify a single 128-sample IQ (in-phase and quadrature) observation window from the 6dB subset of RadioML2016.10a into one of 11 analog and digital modulation classes (8PSK, AM-DSB, AM-SSB, BPSK, CPFSK, GFSK, PAM4, QAM16, QAM64, QPSK, WBFM), an 11-way problem. Random-guess accuracy is 1/11, about 0.091.

All tabular models receive the same 28-dimensional physical feature vector, drawn from four groups. (1) IQ imbalance and DC offset, artifacts left by the receiver’s analog front end and a phase-symmetry indicator that varies by modulation type. (2) Statistical moments of instantaneous amplitude, phase, and frequency (standard deviation, skewness, kurtosis); digital modulations step abruptly in amplitude or phase while analog modulations vary continuously, so these moments discriminate classes strongly. (3) Swami-Sadler style higher-order cumulants (combinations of second, fourth, and sixth order), classical discriminative features designed in modulation theory to differ by class. (4) Spectral features (per-band power, spectral centroid, flatness, entropy, spread, carrier frequency offset, and peak-to-average power ratio), the shape a modulation leaves in the frequency domain. These features are cheap enough to compute thousands of windows per second and feed directly into a classifier after standard scaling.

We compared five families of models. Gradient-boosted trees: LightGBM, XGBoost, and CatBoost. Bagged trees: Random Forest, as a control. Deep tabular architectures: RealMLP and TabM, both from the pytabkit implementation, and a retrieval-based tabular model, TabR. A pretrained tabular foundation model, TabPFN v2, which we note natively supports at most 10 classes, so we wrapped it in a one-versus-rest scheme for this 11-way problem. To see the ceiling of ensemble stacking, we ran AutoGluon’s tabular predictor with a 300-second time limit. Finally, as a deep learning control with no feature engineering at all, consuming the raw 128-sample IQ window directly, we used our own reproduction (following the paper’s description, without reference to official code) of a mixture-of-experts transformer approach to modulation classification: a convolutional patch embedding, four transformer encoder layers, a top-2-routed mixture-of-experts feed-forward block in place of a standard FFN, and roughly 1.39 million parameters. This deep model is not tabular, and, unlike every other model in the comparison, it must learn its own features directly from the raw signal rather than starting from the 28 physical features. That difference in input representation is exactly the question this paper is trying to answer: the practical gap between a model that receives hand-engineered features and one that has to learn them from data itself.

TabR’s pytabkit implementation installed its extra dependencies (skorch, faiss-cpu) cleanly, but training repeatedly hung during the first epoch’s validation step and had to be force-killed after exceeding a 10-minute limit. We excluded it from the results as a runtime issue specific to this execution environment (Apple Silicon, MPS backend), not a judgment on the model itself, and report it honestly as unmeasured.

4 Data and Experimental Setup

We ran two distinct experiments on the same benchmark (RadioML2016.10a, 6dB), and it is important to keep their purposes separate.

The first is the model-family comparison (Table 1 in Section 4), using a single fixed split of 539 training examples and 1,100 test examples across every model listed above. On this split, LightGBM’s accuracy of 0.855 held stable across five different random seeds, ranging from 0.8527 to 0.8564 with a mean of 0.8535, confirming it is not a single-seed fluke.

The second is the data-scaling sweep (Table 2 in Section 4), which asks a different question. Training set size N is grown across 539, 2,000, 5,000, and 9,900, against the same fixed test set (1,100 examples, identical for every N), to see how the gap between GBM and the deep model changes with data volume. For each N , we used stratified sampling to keep per-class counts nearly even, and trained the deep model (mixture-of-experts transformer) on a single GPU for 600 epochs, learning rate 0.001, dropout 0.15, label smoothing 0.05, and weight decay 0.01. Even where this second experiment’s N matches the first experiment’s size (539), it is drawn as a separate resample, so the two experiments’ LightGBM numbers do not agree exactly (0.855 in Table 1 versus 0.728 at $N=539$ in Table 2). The two tables are two different cuts of the same question, and should be read for their relative rankings and trends within each table rather than compared against each other in absolute terms.

At 5,000 and 9,900 training examples, we obtained GBM results, but the corresponding deep model training runs at those sizes did not complete (the jobs were recorded as cancelled or lost). This is an explicit gap in this paper, and Section 5 addresses its implications directly.

5 Results

5.1 Table 1: Accuracy by Model (539 training examples, on 28 physical features, 1,100 test examples)

Model	Family	Accuracy (11-way)	Note
LightGBM	Gradient boosting	0.855	Best
TabPFN v2	Pretrained tabular foundation model	0.848	Required one-versus-rest wrapping (native 10-class cap)
XGBoost	Gradient boosting	0.848	
CatBoost	Gradient boosting	0.848	
Random Forest	Bagged trees	0.840	
AutoGluon	Stacked ensemble	0.818	Stacking gave no gain on this feature set
TabM	Deep tabular	0.781	
RealMLP	Deep tabular	0.778	
Raw-IQ deep learning (reproduction)	Transformer (MoE)	0.681	No feature engineering, raw 128-sample IQ input
TabR	Deep tabular (retrieval-based)	Unmeasured	Skipped due to a training deadlock
Random guess	-	0.091	11-way baseline

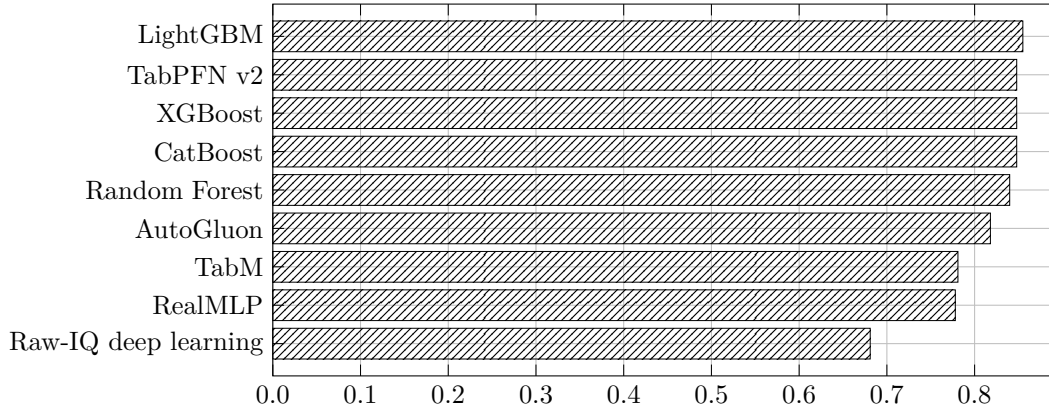


Figure 1: Accuracy by model

The most striking feature of this small-data regime is how tightly the top ranks cluster. The three gradient-boosting variants (LightGBM, XGBoost, CatBoost) and the pretrained foundation model TabPFN v2 sit within a band of 0.840 to 0.855, with Random Forest immediately behind them. AutoGluon’s stacked ensemble, which combines several models, actually landed 3.7 percentage points below a single LightGBM, suggesting that averaging out individual model variance does not pay off much on this feature set at this data size. Deep tabular architectures (TabM, RealMLP), despite receiving the same 28 features, trailed the tree ensembles by 6 to 8 percentage points. That is an important signal: it suggests the gap between tabular neural networks and tree ensembles here is less about “missing features” and more about “not enough data.” Finally, the deep model that learns directly from raw signals with no feature engineering came in last at 0.681, a full 17.4 percentage points behind the top performer, LightGBM.

5.2 Table 2: The GBM-Deep Learning Gap by Data Scale

Training size N	LightGBM	XGBoost	Raw-IQ deep learning	GBM advantage (pp)
539	0.728	0.719	0.368	36.0
2,000	0.794	0.780	0.581	21.3
5,000	0.814	0.817	Incomplete (job failed)	-
9,900	0.844	0.831	Incomplete (job failed)	-

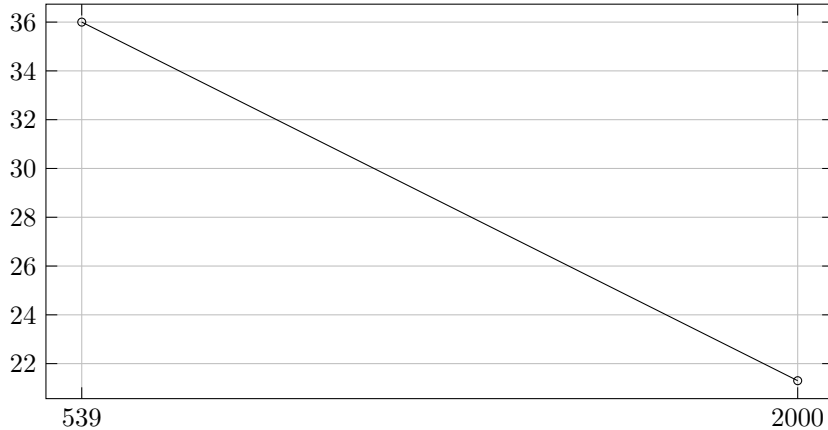


Figure 2: GBM advantage

Looking only at the two points we secured, the trend is unambiguous. Growing the training set from 539 to 2,000 examples, roughly 3.7 times larger, pushed the raw-IQ deep model’s accuracy from 0.368 to 0.581, a jump of 21.3 percentage points. Over the same interval, LightGBM rose only 6.6 percentage points, from 0.728 to 0.794. As a result, GBM’s advantage nearly halved, from 36.0 to 21.3 percentage points. The deep model’s learning curve is clearly far steeper than the GBM curve over this range, even with only two points.

Per-class detail shows this narrowing is not uniform across classes. The raw-IQ deep model’s per-class recall jumped substantially for some modulations: AM-SSB rose from 0.51 to 0.87, BPSK from 0.25 to 0.79. Other classes lagged: QAM16 rose only from 0.14 to 0.24, and QAM64 from 0.17 to 0.30. A reasonable explanation is that higher-order constellations require distinguishing finer distances between constellation points, which demands more data. Supplementary metrics we also examined improved similarly: a binary (real versus circular constellation) discrimination accuracy rose from 0.714 to 0.905, and a three-way order discrimination (BPSK/QPSK/8PSK) rose from 0.19 to 0.47, though the latter remained well below a classical signal processing baseline (cyclostationary features with sufficient integration, roughly 0.8) on the same data.

The gap in training cost is far larger than the gap in accuracy. In this sweep, LightGBM took 8.15 seconds at 539 training examples and 43.71 seconds at 9,900, both on a single CPU core. The raw-IQ deep model took roughly 3,200 seconds (53 minutes) to train on 539 examples over 600 epochs, and roughly 11,240 seconds (3 hours 7 minutes) at 2,000 examples, both on a single GPU. The gap in training cost widens faster than the gap in accuracy narrows, which nearly settles the practical choice at this data scale on its own.

6 Limitations: The Crossover Needs to Be Revisited at Larger Scale

Stated honestly and narrowly, this paper’s conclusion is the following. In a low-data regime of hundreds to a few thousand training examples, on 28-dimensional physical features, gradient boosting is the practical winner. But the much stronger claim, that deep learning can never surpass GBM no matter how much data is added, is not supported by this experiment. There are three reasons.

First, the observed gap-narrowing trend (36.0 to 21.3 percentage points) rests on exactly two data points. Extrapolating a line through those two points out to 5,000 or 9,900 examples is not statistically justified. We in fact attempted deep model reruns at both of those sizes, and neither run completed, so we cannot even confirm whether a crossover point exists within this dataset’s range (up to 9,900 examples) until that gap is filled.

Second, this is not a pure architecture-versus-architecture comparison. The GBM family starts with an information advantage in the form of 28 physical features engineered by hand, while the raw-IQ deep model has to learn that feature extraction step itself from data. The fact that deep tabular models given the same 28 features (TabM, RealMLP) still trailed GBM by 6 to 8 percentage points in this paper suggests that much of the gap is less about “neural networks are worse” and more about “at this amount of data, the neural network family as a whole has not yet made full use of it.” This experiment did not test the condition of giving the deep model both the same physical features and substantially more data at once.

Third, this experiment is confined to a single SNR (6dB) on a single public simulated dataset. Whether the ranking holds under real channel distortion, hardware-specific variation, or mixed- SNR conditions has to be verified separately. It would also be inappropriate to compare this experiment’s numbers directly to the figures reported by the original paper behind the raw-IQ deep model’s methodology (evaluated on the full RadioML dataset, averaged across multiple SNRs), since that paper trains on thousands of examples per class, far larger than the deliberately small, low-data regime this paper starts from (49 examples per class at the smallest point).

The claim this paper actually supports, then, is this: for signal classification tasks where physical features can be extracted, if training data is limited to a few thousand examples or fewer, choosing tabular boosting as the default is reasonable. Whether that advantage holds as data scale grows much larger is an open question that requires further experimentation; the trend observed so far points toward a narrowing gap.

7 Reproducibility

This experiment can be built entirely from public assets. The dataset is the 6dB SNR subset of RadioML2016.10a (O’Shea and West, 2016), consisting of 128-sample IQ observation windows with class labels. The 28-dimensional physical feature set is a combination of standard signal processing operations: IQ imbalance and DC offset estimation, statistical moments of instantaneous amplitude, phase, and frequency, second-, fourth-, and sixth-order Swami-Sadler cumulants, and band power, spectral moments, carrier frequency offset, and peak-to-average power ratio. These features can be reproduced directly from definitions published in standard signal processing references.

Every tabular model used here is open source: LightGBM, XGBoost, CatBoost, scikit-learn’s Random Forest, pytabkit’s RealMLP, TabM, and TabR, TabPFN v2, and AutoGluon-Tabular. All were trained with default hyperparameters and no additional tuning, deliberately reflecting the most common starting point in practice; deeper tuning could change the ranking, and that possibility remains open.

The raw-IQ deep learning control is our own independent reproduction, following the description in a recently published mixture-of-experts transformer approach to modulation classification (arXiv:2606.09085), without reference to official code and without any intent to reproduce that paper’s reported numbers. The architecture is a convolutional patch embedding, four transformer encoder layers (model dimension 128, 4 heads), a mixture-of-experts feed-forward block with 4 experts and top-2 routing in place of each layer’s standard feed-forward network, totaling roughly 1.39 million parameters. Training used an Adam-family optimizer, learning rate 0.001, dropout 0.15, label smoothing 0.05, weight decay 0.01, and 600 epochs. Applying the same hyperparameters to different training set sizes (539, 2,000) produces the sweep in Table 2.

For the data-scaling sweep, training examples for each N were drawn by stratified random sampling from the full training pool with near-even per-class counts, and the 1,100-example test set was kept fixed across every N to preserve comparability. Reproducing this design in full requires only one public dataset, standard signal processing feature-extraction code, and the open-source libraries listed above; specialized hardware is not required for training the gradient-boosted models. Only the deep learning control requires a single GPU.

8 References

1. T. O'Shea, N. West, "Radio Machine Learning Dataset Generation with GNU Radio," Proceedings of the GNU Radio Conference (GRCon), 2016. (RadioML2016.10a)
2. G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," Advances in Neural Information Processing Systems (NeurIPS), 2017.
3. T. Chen, C. Guestrin, "XGBoost: A Scalable Tree Boosting System," Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016.
4. L. Prokhorenkova et al., "CatBoost: Unbiased Boosting with Categorical Features," Advances in Neural Information Processing Systems (NeurIPS), 2018.
5. N. Hollmann et al., "Accurate Predictions on Small Data with a Tabular Foundation Model," Nature, 2025. (TabPFN v2)
6. D. Holzmüller et al., "Better by Default: Strong Pre-Tuned MLPs and Boosted Trees on Tabular Data," Advances in Neural Information Processing Systems (NeurIPS), 2024. (RealMLP)
7. N. Erickson et al., "AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data," arXiv:2003.06505, 2020.
8. arXiv:2606.09085, mixture-of-experts transformer approach to modulation classification (reproduced from the paper's description, without reference to official code).
9. A. Swami, B. Sadler, "Hierarchical Digital Modulation Classification Using Cumulants," IEEE Transactions on Communications, 2000. (theoretical basis for the higher-order cumulant features)